

Week 1 Interval Operations Report

Student Name: Enis Ögdüm

1. Results of Operations

Given the intervals $X = [-1, 2]$ and $Y = [3, 5]$, the operations resulted in: $X + Y = [2, 7]$ $X - Y = [-6, -1]$ $X * Y = [-5, 10]$ **width of X = 3** **midpoint of X = 0.5**

2. Interpretation of the Multiplication Result

Interval multiplication calculates the range covering all possible products of any number from interval X and any number from interval Y. In this case, multiplying numbers from $[-1, 2]$ and $[3, 5]$ yields a minimum possible product of -5 (from -1×5) and a maximum possible product of 10 (from 2×5), resulting in the final interval boundary of $[-5, 10]$.

3. Explanation of Midpoint and Width

- **Midpoint:** The midpoint is the exact center or average value of the interval's two endpoints. In practical applications, it is often used as the "expected value," best estimate, or most representative single number for the interval.
 - **Width:** The width is the total length or span of the interval, calculated as the difference between the upper and lower endpoints. It effectively measures the degree of uncertainty, tolerance, or margin of error contained within that interval.
-

Script: 3.py

1. Results of Operations

Given the interval $X = [1, 2]$ and the functions $f1(X) = X * X - X$ and $f2(X) = X * (X - [1, 1])$:
 $f1(X) = [-1, 3]$; **width = 4** $f2(X) = [0, 2]$; **width = 2**

2. Answers to Questions

1. **Are the two results identical?** No, the results are different. $f1(X)$ evaluates to $[-1, 3]$ and $f2(X)$ evaluates to $[0, 2]$.
2. **Which result is narrower?** The result from $f2(X)$ is narrower. It has a width of 2, compared to the width of 4 from $f1(X)$.
3. **Why can two equivalent formulas produce different interval estimates?** This occurs due to the "**dependency problem**" in interval arithmetic. When a variable like X appears multiple times in a formula (as in $X * X - X$), interval mathematical operations treat each occurrence of X as independent. It calculates the maximum possible boundary by assuming the first X is at its highest extreme, while the second X is simultaneously at its lowest extreme. By factoring the equation into $X * (X - 1)$, we minimize how many independent times X appears. This reduces the artificial worst-case bounding and produces a much tighter, more accurate interval estimate.

Script: 4.py

1. Results of Operations

Given the functions $f1(X) = X * X - X$ and $f2(X) = X * (X - [1, 1])$, we evaluate them across three increasingly narrow intervals: * **For $X = [1, 2]$:** * $f1(X) = [-1, 3]$; width = 4 * $f2(X) = [0, 2]$; width = 2 * **For $X = [1, 1.5]$:** * $f1(X) = [-0.5, 1.25]$; width = 1.75 * $f2(X) = [0, 0.75]$; width = 0.75 * **For $X = [1, 1.1]$:** * $f1(X) \approx [-0.10, 0.21]$; width ≈ 0.31 * $f2(X) = [0, 0.11]$; width = 0.11

2. Answers to Discuss Questions

1. **What happens to the width of the result when the input interval is narrowed?**
As the input interval X gets narrower (its width decreasing from $1 \rightarrow 0.5 \rightarrow 0.1$), the output widths for both $f1$ and $f2$ also shrink significantly. The result becomes much more precise.
2. **Does the difference between $f1$ and $f2$ decrease or increase?** The absolute difference in width between $f1$ and $f2$ **decreases** as the input interval narrows. When $X = [1, 2]$ the difference in width is 2. When $X = [1, 1.5]$ the difference drops to 1. When $X = [1, 1.1]$ the difference is only 0.20.
3. **Why is this important in practical numerical computations?** This demonstrates a core technique in numerical computing: if the “dependency problem” causes too much error, subdividing a large input interval into smaller sub-intervals drastically tightens the resulting bounds. It proves that to compute highly accurate global bounds for complex functions, algorithms should recursively chop inputs into smaller intervals rather than evaluating the entire space at once.

Script: 5.py

1. Sequence of Intervals Obtained

Starting with the initial interval $X = [1, 2]$, the Newton iterations resulted in the following bounds: * **Iteration 1:** $X = [1.375, 1.4375]$, width = 0.0625 * **Iteration 2:** $X = [1.4140625, 1.4144176136363635]$, width ≈ 0.000355 * **Iteration 3:** $X = [1.4142135592945242, 1.4142135659471788]$, width $\approx 6.65e-09$ * **Iteration 4:** $X = [1.4142135623730951, 1.4142135623730951]$, width = 0.0 * **Iteration 5:** $X = [1.414213562373095, 1.414213562373095]$, width = 0.0

2. Observation on Interval Contraction

The interval sharply contracts with each successive iteration. By simply looking at the width trend ($0.0625 \rightarrow 0.0003 \rightarrow 0.000000006 \rightarrow 0.0$), we can see that the contraction is quadratic. In precisely 4 steps, the bounds collapse tightly to 0.0 , perfectly converging exactly on the positive root ($\sqrt{2}$).

3. Usefulness in Verified Numerical Computations

The Interval Newton method is powerful for verified numerical computations because it yields absolute mathematical guarantees. Instead of standard algorithms that spit out a single floating-point number with an unknown confidence margin, interval operators dynamically trap the exact solution between guaranteed upper and lower limits. If the interval width rapidly converges to a tight number, like it did here, the method mathematically proves that a unique true root exists and establishes guaranteed error bounds covering rounding inaccuracies.

Script: 6.py

1. Results of Operations

Given the measurements of a rectangle with tolerances as intervals $L = [0.999, 1.001]$ and $W = [1.999, 2.001]$, computing the possible bounds of its area yields: * **Area A** = $[1.997001, 2.003001]$ * **Width of the result** ≈ 0.006

2. Final Questions

1. **What does the resulting interval for the area mean?** The resulting interval $[1.997001, 2.003001]$ represents the strictly guaranteed tolerance limits (the absolute smallest and largest possible bounds) for the area of the rectangle, calculated by combining the measurement uncertainties natively present in both the length and width intervals.
 2. **Why does the single value $A = 2$ not convey the full information?** Relying on a single nominal value like $A = 2$ blindly assumes perfect 1×2 manufacturing precision. It completely hides the fact that the actual physical area could shrink to ~ 1.997 or swell to ~ 2.003 due to compounding tolerances. If this component needs to fit tightly within a precision assembly, ignoring this variance information could lead to catastrophic jamming or loose connections.
 3. **In which engineering or scientific applications could this way of computing be useful?** This method is primarily essential in **Tolerance Stack-Up Analysis** found in mechanical engineering and precision manufacturing (such as aerospace or automotive engineering). It is also incredibly useful in bounding robotic kinematics, determining structural safety load margins, and orbital mechanics, where researchers must rigorously prove that compounding physical/environmental variances will still strictly result in a safe and bounded operational state.
-

Final Summary

Interval analysis provides absolute mathematical guarantees for numerical computations by strictly trapping calculated results within rigorous upper and lower bounds, completely eliminating the hidden uncertainties of standard floating-point approximations. The main advantage of this method is its ability to natively track strict margins of error and prove the

existence of solutions, bringing absolute reliability to algorithms. However, its primary limitation is the “dependency problem,” where significant overestimation predictably occurs because interval arithmetic operations blindly treat multiple occurrences of the exact same variable as entirely independent. For example, in an unsimplified equation like $x^2 - x$, the method calculates an artificially wide “worst-case scenario” assuming one x is at its maximum while the identically valued x is simultaneously at its minimum. Despite this computational overestimation, interval methods remain profoundly useful in critical real-world practices where safety margins are mandated. It is extensively utilized in structural engineering load analysis, robotic kinematics, aerospace trajectory tracking, and manufacturing tolerance stack-ups, where engineers must absolutely guarantee that compounded physical variances stay within safe, operational bounds without risking catastrophic mechanical failure.